



SCHOOL OF MATHEMATICS AND APPLIED SCIENCES
DEPARTMENT OF COMPUTING

TO : MR. S. YUTE

FROM : MR. MATTHEWS A. MASANDA & MR. LYSON R. MBUGHI

REGISTRATION NO : BSC-COM- NE-16-23 & BSC-COM-NE-20-23

COURSE NAME : NETWORK PROGRAMMING AND APPLICATION DEVELOPMENT

COURSE CODE : NET322

ASSIGNMENT TITLE : IOT TELEMETRY PIPELINE & DASHBOARD

DUE DATE : 8TH MAY, 2026

NET322: Network Programming & Applications Development

Practical Test 2 — Assignment 1: Theory Paper

IoT Telemetry Pipeline & Dashboard

Serialization Choices

Different serialization formats are used in the telemetry system because each component has different requirements regarding readability, performance, and interoperability. YAML is used for configuration files because it is highly human-readable and easy to edit manually. Sensor configurations such as sensor IDs, intervals, and network addresses are typically maintained by developers or administrators, so readability is more important than transmission efficiency. YAML also supports nested structures with very little syntax overhead, making it less verbose than XML while remaining understandable.

Protocol Buffers (Protobuf) are used on the sensor-to-server communication link because they provide compact binary serialization with strong schema support and high performance. In an IoT environment where many sensors may continuously send telemetry data, minimizing bandwidth usage and serialization overhead is important. Protobuf messages are significantly smaller and faster to serialize and deserialize compared to text-based formats such as JSON or XML. The .proto schema also ensures that both the sensors and server share a consistent data structure.

For the public REST API, JSON and XML are more suitable because they are widely supported by external clients. JSON is lightweight, easy to parse, and commonly used in modern web and mobile applications. XML, although more verbose, provides stronger document structure and compatibility with older enterprise systems. Supporting both formats through content negotiation allows the system to serve modern applications and legacy tools simultaneously.

Web Services Protocol

REST is the most appropriate web service architecture for the historical-data API because it is lightweight, resource-oriented, and naturally integrates with HTTP. In this system, resources such as sensors and readings can be represented using clear URI structures like `/sensors` and `/sensors/{id}/readings`. REST also uses standard HTTP methods such as GET, POST, and DELETE, making the API predictable and easy to use.

Another important advantage of REST is statelessness. Each request contains all the information needed for processing, allowing the server to handle many concurrent clients efficiently. This design fits well with scalable telemetry systems where clients may frequently connect and disconnect.

SOAP differs from REST because it is a strict protocol that relies heavily on XML messaging and additional standards. Although SOAP provides strong formal contracts and advanced enterprise features, it introduces higher complexity and verbosity, which are unnecessary for a lightweight telemetry system. RPC focuses on invoking remote functions directly rather than manipulating resources. While RPC can simplify internal service communication, it is less aligned with web-oriented resource management and standard HTTP semantics. REST therefore provides the best balance of simplicity, scalability, and interoperability for the proposed system.

AsyncIO vs Threading

The telemetry system is primarily an I/O-bound application because most operations involve waiting for network communication rather than performing heavy CPU computations. The server continuously waits for incoming sensor data, REST requests, and WebSocket messages. During these waiting periods, CPU usage remains relatively low.

AsyncIO is well suited for this type of workload because it uses coroutines and an event loop to manage many concurrent connections efficiently within a single thread. Instead of blocking while waiting for I/O operations to complete, coroutines yield control back to the event loop, allowing other tasks to execute. This approach reduces memory overhead and context-switching costs compared to creating large numbers of operating system threads.

Threading would be more appropriate in situations involving blocking libraries that do not support asynchronous execution, or when integrating with legacy synchronous code. Processes would be preferable for CPU-intensive workloads such as image processing, machine learning, or complex mathematical computations because they can fully utilize multiple CPU cores. Since the telemetry system mainly performs network communication and lightweight serialization, AsyncIO provides a more scalable and efficient concurrency model.

HTTP Mechanics

Content negotiation is an important HTTP mechanism used in the REST API design. Clients specify their preferred response format using the Accept header, for example application/json, application/xml, or application/yaml. The server then serializes the response into the requested format. This allows multiple client types to interact with the same API while maintaining compatibility with different technologies and legacy systems.

The WebSocket upgrade handshake is another critical mechanism in the system. Initially, the client establishes a normal HTTP connection and sends an upgrade request to switch protocols from HTTP to WebSocket. After the handshake succeeds, the connection becomes a persistent full-duplex communication channel. This allows the server to push live sensor readings to connected dashboard clients in real time without requiring repeated polling requests. The WebSocket approach reduces latency and network overhead, making it suitable for live telemetry streaming.

References

- Ben-Kiki, O., Evans, C., & döt Net, I. (2021). YAML Ain't Markup Language (YAML) version 1.2.3. <https://yaml.org/spec/1.2.3/>
- Berners-Lee, T., Fielding, R., & Frystyk, H. (1999). Hypertext Transfer Protocol — HTTP/1.1 (RFC 2616). IETF. <https://www.rfc-editor.org/rfc/rfc2616>
- Fette, I., & Melnikov, A. (2011). The WebSocket Protocol (RFC 6455). IETF. <https://www.rfc-editor.org/rfc/rfc6455>
- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Doctoral dissertation, University of California, Irvine]. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Google. (2023). Protocol Buffers documentation. <https://protobuf.dev/>
- Python Software Foundation. (2024). asyncio — Asynchronous I/O. <https://docs.python.org/3/library/asyncio.html>
- Richardson, L., & Ruby, S. (2007). RESTful web services. O'Reilly Media.